Lecture Notes

MVC Architecture, Code Organisation and Authentication Flow in Express.js

Instructor: Abhishek Purohit | Date: 29 Oct 2026 | Prepared for: Sumeet Sharma

What this lecture covers (in teaching order)

1. Why code must be separated into concerns (house analogy) and why **index.js** should not hold everything
2. The **MVC** design pattern: Model, View, Controller
3. Project folder structure under **src/**: controllers, configs, routers, middlewares, models
4. Moving request/response logic into **controllers** (welcome, product) with naming conventions
5. Moving database setup into a **configs/DbConfig.js** file
6. Further layers in enterprise apps: **service** and **DAO**; single responsibility principle
7. Moving routes into **routers** using **express.Router()**
8. **export default** vs named export, and how to import each
9. Registering routers with **app.use()** after **express.json()**
10. Request flow diagram, and where **middleware** fits
11. API security: why authentication is needed, **login + token** flow, self-service restaurant analogy
12. Admin table and **admin registration API** (controller + router), testing in Postman
13. Why plain-text passwords are dangerous (hashing comes next)

How to read these notes

- Normal text and yellow-free sections are taken from the lecture, converted from spoken words into clear written English.
- Boxes labelled **[Extra - not from the lecture]** are my additions (extra explanation, interview material, technical clarifications). Please treat them as supplementary.
- Green boxes show **expected output**. Grey boxes show **code**.
- The transcript was auto-generated and is partly garbled (mixed Hindi/English). Where a sentence was unclear I used the meaning that the code and the surrounding explanation make certain.

Note: The lecture did **not** show the full original or final **index.js** as a file. The index.js versions in these notes are **reconstructed** from what the instructor described (imports, **express.json()**, **app.use(...)**, port 7500). All controller, router and DbConfig code is exactly the code you supplied.

1. Separation of Concerns

1.1 The house analogy

The instructor started with a real-life comparison. Inside a home, you keep kitchen utensils in the kitchen, clothes in the wardrobe, and vehicles in a separate parking area. We separate things by their purpose in every domain of life. We do not mix everything and dump it in one place.

Coding is the same. In the coding world it is also required to separate different concerns. A **concern** is one kind of job in your application, for example: accepting requests, running business logic, talking to the database, or reading configuration.

1.2 What changes and what does not

- **Output / behaviour does not change.** Your back end behaves exactly as before; the same APIs return the same results.
- **Only the way of writing code changes.** The same amount of code is now spread across various files and folders.
- The reason: organised files and folders make the application **more scalable, easier to maintain, and easier to debug**.

1.3 Express.js vs frameworks like NestJS

Express.js does not force any structure on you. You must create this organisation **manually, from scratch**. A framework such as **NestJS** is different: it gives you a complete ecosystem for separation of concerns out of the box. It provides a predefined approach for writing code, and the code/file/folder layout is already set up. (The instructor suggested exploring NestJS later, when you have time.)

1.4 Why not write everything in index.js?

If you keep every route, every database call and all logic in **index.js**, it quickly becomes one huge file. It becomes hard to read, hard to change and hard to debug. To split code in a sensible way you cannot split randomly. There has to be a **fixed design pattern** (a blueprint) to follow. The instructor stressed that this idea is **not technology-specific**: you will use the same thinking in JavaScript, Java, Python or any other language.

2. MVC Design Pattern

The pattern chosen for this course is **MVC** (Model - View - Controller). Analogy: to divide a house you use terms like BHK - bedroom, hall, kitchen. In the same way, MVC is the terminology used to divide the code of an entire application.

Part	Meaning (instructor's explanation)	Example
Model	Represents your **data**, or the **entities** on which you perform CRUD operations and with which the application deals. You can create a class (or interface) so that every developer can recognise the entity's properties and behaviour.	In e-commerce: Customer, Order, Product, Cart - each can be a class.
View	Whatever is **visible to the user**: the user interface, i.e. the front end. It is kept separate and will be built later in React.	Web pages, mobile screens
Controller	A function / piece of logic that **accepts the request and generates the response**. That is its only job. Every back end must have controllers.	Functions that earlier were written inside app.get / app.post

Note: In this lecture the instructor first focuses on the **Controller** (back-end side). The **Model** layer (for example a Product class with name, price, description, qty) was explained as an idea - 'ideally we should create it' - but was not coded in this session. The View is the front end, handled later.

[Extra - not from the lecture] Entity = a real-world thing your application stores data about (a product, a customer, an admin). In a database, one entity usually becomes one table.

3. Folder Structure and the Role of index.js

3.1 What index.js should do (instructor's recommendation)

index.js is only for starting the server. It creates the Express application, starts it, and may register middlewares. It should not contain business logic.

- index.js is the **starting point**. It receives a request and **hands over (delegates)** the job to the right layer.
- Everything else is broken into separate files and folders.
- Inside the project, create one folder named **src**. All the layer folders go inside **src** (recommendation to follow).

3.2 Layer folders

Folder (inside src)	Purpose	File-naming rule given
`controllers`	Functions that accept the request and generate the response	One controller file **per entity**: ProductController.js, AdminController.js, WelcomeController.js
`configs`	Configuration: database, file system, anything else configurable	One file per kind of config: DbConfig.js
`routers`	Routing: which URL + method calls which controller function	One router file **per entity**: ProductRouter.js, AdminRouter.js, WelcomeRouter.js
`middlewares`	Middleware functions (checks before the controller runs)	Created later (authentication)
`models`	Entity representation (classes)	Idea explained; not coded in this lecture

3.3 Final structure after this lecture

```
project-root/
|-- index.js                    <- only creates the app, registers middleware/routers, starts server
|-- package.json
`-- src/
    |-- configs/
    |   `-- DbConfig.js         <- MySQL connection pool
    |-- controllers/
    |   |-- WelcomeController.js  <- base URL "/"
    |   |-- ProductController.js  <- save, getAll, getById, deleteById
    |   `-- AdminController.js    <- save (admin registration)
    `-- routers/
        |-- WelcomeRouter.js
        |-- ProductRouter.js
        `-- AdminRouter.js
```

[Extra - not from the lecture] Because the code uses **import ... from** (ES modules), your **package.json** must contain **"type": "module"**, and every relative import must end with **.js** (for example ../configs/DbConfig.js). Forgetting the extension is a common error in Node ES modules.

4. Controllers

4.1 What is a controller?

A controller is a function that accepts the request and generates the response. That is it.

All the functions you wrote earlier inside **app.get** and **app.post** that take **request** and **response** arguments are controller functions. We now move them out of index.js into a separate layer: the **controllers** folder.

4.2 One controller file per entity

Do not put all controllers in one file. Create one controller per entity:

- Product entity -> **ProductController.js** (only product-related request/response functions).
- Customer entity -> a CustomerController.js (only customer functions).
- Admin entity -> **AdminController.js**.
- Order, cart etc. -> their own controller files.

Benefit: if you later need to add a feature to customers, you write code only in the customer controller. If something about admin requests/responses must change, it goes only into the admin controller. This is **separation of concerns** and **modularity**: well organised, not mixed in a single place.

4.3 The base URL needs its own controller

The function that handles the base URL **"/"** is not about the product entity, so it should not go into the product controller. The instructor named it the **welcome controller** (you could also call it home controller or front controller). Each function stays in the controller where it belongs by concern.

5. Refactoring Step by Step

Phase 1 - Initial approach: everything inside index.js

At the start of the lecture, **index.js** contained: the MySQL connection pool creation, the base URL handler, and every product route with its handler (save, get all, get by id, delete by id) written inline with **app.get / app.post / app.delete**.

[Extra - not from the lecture] A typical shape of that starting code (reconstructed; the original was not provided):

```
// index.js  (BEFORE - everything in one file, reconstructed outline)
import express from "express";
import { createPool } from "mysql2/promise";

const sqlConnectionPool = createPool({ /* host, port, user, password, database ... */ });

const app = express();
app.use(express.json());

app.get("/", (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
});

app.post("/product", async (request, response) => { /* insert query ... */ });
app.get("/product", async (request, response) => { /* select all ... */ });
app.get("/product/:id", async (request, response) => { /* select by id ... */ });
app.delete("/product/:id", async (request, response) => { /* delete by id ... */ });

app.listen(7500, () => console.log("Server started"));
```

Phase 2 - Problems with this approach

- Index.js mixes many concerns: server start, DB setup, routing, request handling, SQL.
- With hundreds of APIs for product, customer, order, cart, etc., the file becomes huge and unmaintainable.
- Changing one thing (for example admin behaviour) means searching through a large shared file; a small edit can break something unrelated.
- Hard to debug and hard for a team to work on without conflicts.

Phase 3 - Improved approach: move code into layers

Step 3.1 - WelcomeController

Purpose: handle the base URL. **Problem solved:** take this handler out of index.js. You may write it as an arrow function or with the **function** keyword (choice is yours). The instructor named it **welcome**.

```
// src/controllers/WelcomeController.js
export const welcome = (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
}
```

- `export` makes the function available to other files. Without it, index.js cannot import it.
- `request` holds what the client sent; `response` is used to reply.
- `response.send({...})` sends the object to the client as JSON.

Now import it in index.js. The path is written **relative to the file doing the import**. index.js sits next to the **src** folder, so: go into src, then controllers, then the file.

```
// index.js
import { welcome } from "./src/controllers/WelcomeController.js";

app.get("/", welcome);      // pass the function REFERENCE - do not write welcome()
```

Common mistake: Writing **app.get("/", welcome())** would **call** the function immediately at start-up. The instructor stressed: *'we do not call these controller functions; we only pass the definition (reference). Express calls it when a GET request arrives on the base URL.'*

Flow now: a GET request arrives on **"/"** -> index.js accepts it -> index.js hands the job to the controller function **welcome** -> the controller accepts the request and generates the response.

Instructor's rule of thumb: whether you build a server in JavaScript, Java, Python or any other technology, think in terms of controllers. The index/starting file only redirects control to different layers.

Step 3.2 - Database configuration: configs/DbConfig.js

Problem: the product functions need the **SQL connection pool**, which was created in index.js. Creating the connection is not the controller's job, and not index.js's job either. It is **configuration** - specifically **database configuration**. Configurations (database, file system, anything else) belong in a separate **configs** folder, one file per configuration. So we create **src/configs/DbConfig.js**.

```
// src/configs/DbConfig.js
import { createPool } from "mysql2/promise";

export const sqlConnectionPool = createPool({
    host: "localhost",
    port: 3306,
    user: "root",
    password: "root",
    database: "productdb",
    connectionLimit: 10,
    queueLimit: 0
});
```

- `createPool` (from mysql2/promise) creates a **pool**: a ready set of database connections that are reused.
- The import of `createPool` is no longer needed in index.js. It moved with the code to DbConfig.js.
- `export const sqlConnectionPool` lets any controller import the pool.

[Extra - not from the lecture] Option meanings: host/port = where MySQL runs (3306 is MySQL's default port); user/password/database = credentials and database name; **connectionLimit: 10** = at most 10 connections in the pool; **queueLimit: 0** = no limit on requests waiting for a free connection. Use real, safe credentials in real projects; do not commit passwords to Git.

Step 3.3 - ProductController (save, getAll, getById, deleteById)

Now move every product handler into **src/controllers/ProductController.js**. Because they use **await**, each function must be **async**. Each is exported. Import the pool at the top:

From controllers we must come out of the controllers folder, so **..** then the **configs** folder, then **DbConfig.js**. The name we import is the exact exported name: **sqlConnectionPool**.

Naming convention (from the lecture): because the controller deals only with the product entity, do **not** repeat the word 'product' in every function name. Write **save**, **getAll**, **getById**, **deleteById** instead of saveProduct, getAllProduct, etc. The same idea applies to database tables: use columns **id, name, price** rather than product_id, product_name, product_price. The entity (file / table name) already supplies the context.

```
// src/controllers/ProductController.js
import { sqlConnectionPool } from '../configs/DbConfig.js';

export async function save(request, response) {
    try {
        // step 1: get data from client {name:"", price: 6677, description:"", qty:8}
        const { name, price, description, qty } = request.body;

        // step 2: create a SQL INSERT query
        // const insertQry = `INSERT into product(name, price, description, qty) VALUES('${name}', ${price}, '$
        // this above approach (concating data directly in the SQL query) is prone to sql injection attack

        const insertQry = `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(insertQry, [name, price, description, qty]);
        console.log(result.insertId);

        response.send({ message: "product inserted", insertedId: result.insertId });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function getAll(request, response) {
    try {
        // step 1: create a select query
        const fetchAllProductsQry = "SELECT * FROM product";

        // step 2: run this query
        const [result] = await sqlConnectionPool.query(fetchAllProductsQry);

        response.send(result);

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function getById(request, response) {
    try {
        // step 1: read id from url parameter
        const { id } = request.params;

        // step 2: create a select query
        const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";

        // step 3: run this query
        const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id]);

        if (result.length > 0) {
            response.send(result[0]);
        }
        else {
            response.status(404).send({ message: `Product with id ${id} not found` });
        }

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
```

```
}

export async function deleteById(request, response) {
    try {
        // step 1: read id from url parameter
        const { id } = request.params;

        // step 2: create a delete query
        const deleteQry = "DELETE FROM product WHERE id = ?";

        // step 3: run this query
        const [result] = await sqlConnectionPool.query(deleteQry, [id]);
        console.log(result);

        response.send({ message: "Product Deleted" });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

Explanation of the code

Piece	What it does and why
`async function`	Marks a function that can use **await**. The database call takes time, so we wait for it without blocking the server.
`try { } catch (error) { }`	If anything fails (for example the DB is down), the catch block logs the error and sends **status 500** with a message. The client always gets a proper response and the server does not crash.
`request.body`	The JSON the client sent. It is readable because **express.json()** middleware is registered in index.js.
`const { name, price, description, qty } = request.body`	**Destructuring**: picks the named properties from the body object.
`VALUES(?,?,?,?)`	**Placeholders**. The values are passed separately as an array: [name, price, description, qty]. The driver inserts them safely. This is a **parameterised query**.
`const [result] = await pool.query(...)`	query returns an array whose first item is the result. We destructure it. For INSERT, **result.insertId** is the new auto-incremented id; for SELECT, **result** is the array of rows.
`request.params`	Values from the URL path. For route **/product/:id**, a request to /product/5 gives **id = '5'**.
`result.length > 0`	For getById: if a row was found, send that single row (**result[0]**); otherwise send **404** with a 'not found' message.
`response.status(500).send(...)`	Sets the HTTP status code, then sends the body.

Expected output

```
POST /product           -> { "message": "product inserted", "insertedId": 7 }
GET  /product           -> [ { "id": 1, "name": "...", "price": ..., "description": "...", "qty": ... }, ... ]
GET  /product/1         -> { "id": 1, "name": "...", ... }
GET  /product/9999      -> 404  { "message": "Product with id 9999 not found" }
DELETE /product/1       -> { "message": "Product Deleted" }
Any failure             -> 500  { "message": "Something went wrong" }
```

Why these outputs appear: each function reads the input (body or URL id), builds an SQL string, runs it on the pool, then converts the DB result into the response. insertedId is read from **result.insertId**; the list comes straight from the SELECT result; 404 appears when the select returns zero rows.

Common mistake: Concatenating user data directly into the SQL string (the commented-out line using ${name}, ${price}) is **prone to SQL injection attacks**. The instructor's code keeps it only as a commented warning and uses placeholders (?) instead.

[Extra - not from the lecture] Observation: **deleteById** always replies 'Product Deleted', even if no row with that id existed (result.affectedRows would be 0). Checking **result.affectedRows** would make it more accurate. This was not discussed in the lecture.

Step 3.4 - Import controller functions into index.js

After moving each function, import it in index.js and pass its reference to the route:

```
// index.js (intermediate stage - reconstructed)
import express from "express";
import { welcome } from "./src/controllers/WelcomeController.js";
import { save, getAll, getById, deleteById } from "./src/controllers/ProductController.js";

const app = express();
app.use(express.json());

app.get("/", welcome);
app.post("/product", save);
app.get("/product", getAll);
app.get("/product/:id", getById);
app.delete("/product/:id", deleteById);

app.listen(7500, () => console.log("Server started on 7500"));
```

Several functions can be imported together inside one pair of curly braces separated by commas. The instructor ran **node index.js**, opened Postman and sent a **GET** request to the products URL on port **7500**. All product data came back, so the layered setup worked. (If an import or export were wrong, Node would report that error at start-up.)

Where we are: index.js now accepts requests and delegates to controllers. Controllers accept requests and generate responses. But index.js still contains the routing list. That is the next improvement.

Phase 4 - Further improvement: separate Router layer

Problem: with hundreds of routes (APIs), index.js would again become large and hard to maintain. **Routing must be handled by its own layer**, called the **router**. Create a **routers** folder inside src. Rule: **one router file per entity** (product, customer, admin, cart, order ...). Names can be product routes / product router - the choice is yours. The lecture used **ProductRouter.js**, **WelcomeRouter.js** and later **AdminRouter.js**. The base URL route also gets its own file (welcome router).

Why we cannot use `app` in a router file

app.get / app.post use the **app** variable created in index.js. A router file does not have access to that variable. So Express gives you a dedicated **Router** object. You import the **Router** function from express and call it; it returns a router object. On that object you call **.get, .post, .put, .delete** just as you did on app.

	app (express())	Router (express.Router())
Where it lives	index.js, created once	Any router file, one per entity
Created by	`const app = express()`	`const productRouter = Router()`
Methods	get, post, put, delete, use, listen	get, post, put, delete (no listen)
Purpose	The whole application; start the server	Group routes of one entity in a separate file

WelcomeRouter.js

```
import { Router } from 'express';
import { welcome } from '../controllers/WelcomeController.js'

const welcomeRouter = Router();

welcomeRouter.get("/", welcome);

export default welcomeRouter;
```

- `import { Router } from 'express'` - the function that creates a router object.

- `Router()` - call it to get the router object; we name it welcomeRouter.
- `welcomeRouter.get("/", welcome)` - map GET on the base URL to the controller function.
- The controller is imported **here in the router**, no longer in index.js. Path: from routers, go out one folder with **../**, then **controllers**.
- `export default welcomeRouter` - makes the router available to index.js (explained in section 6).

ProductRouter.js

```
import { Router } from "express";
import { save, getAll, getById, deleteById } from "../controllers/ProductController.js";

const productRouter = Router();

productRouter.post("/product", save);

productRouter.get("/product", getAll);

productRouter.get("/product/:id", getById );

productRouter.delete("/product/:id", deleteById);

export default productRouter;
```

The lines that were in index.js (app.post, app.get, app.delete) now live here, with **app** replaced by **productRouter**. Index.js no longer needs to import the controllers.

Now the responsibilities are: **welcome router** handles the base URL; **product router** handles product routing; index.js receives the request and passes it onwards. It is the router that decides which controller function runs.

6. export default vs Named export

This part of the lecture answers: why did we write **export default productRouter** at the bottom, and why can we import it without curly braces?

6.1 The instructor's explanation

Code executes line by line. If you write the export at the point of declaration, for example `export const welcomeRouter = Router();` at line 4, and then define the routes at line 6 onwards, the instructor's point is that the export statement happens **before** the routing is defined, so you would be exporting the router 'without any routings'.

He explained the idea with a demo file. Imagine **demo.js** declares a variable, runs some logic, and the logic gives the variable its final value (for example 100):

```
// demo.js
let obj = 0;          // initial value 0 (could also be null / undefined)

// ... some logic runs here and finally:
obj = 100;

export default obj;   // exported at the BOTTOM, after the final value is ready
```

> There are cases where you declare a property first and cannot export at that same moment, because some logic has to run first and then the property gets its value. For such cases, put **export default** at the bottom of the file.

The same pattern applies to routers: **first declare** the router object, **then define all routings**, and **at the bottom** write **export default welcomeRouter;** (and **export default productRouter;** for the product router).

6.2 Importing an export-default property

While importing a property exported with export default, we do not write the curly braces, and we do not have to write the exact property name.

```
// named export / named import  (curly braces + exact name)
export const sqlConnectionPool = createPool({ ... });
import { sqlConnectionPool } from "../configs/DbConfig.js";

// default export / default import  (no braces, any name)
export default productRouter;
import productRouter from "./src/routers/ProductRouter.js";
import anyNameYouLike from "./src/routers/ProductRouter.js";   // also valid
```

Why this is allowed: a file that uses export default exports only a **single** property, so there is no confusion about which one you mean. That is why you may refer to it with any name of your choice (same name or different; the value is the same). It is not hard and fast that the name must match - but a matching, meaningful name is good convention.

Point	Named export	export default
Syntax (export)	`export const x = ...;`	`export default x;`
Syntax (import)	`import { x } from './f.js'`	`import anyName from './f.js'`
Curly braces on import	Required	Not used
Name on import	Must be the exact exported name	Any name you want
How many per file	Many	Only one
Used in this lecture	Controller functions, sqlConnectionPool	welcomeRouter, productRouter, adminRouter

[Extra - not from the lecture] Technical clarification. The practical rule 'declare, configure, then export default at the bottom' is a good, always-safe habit, and it is the convention taught in this lecture. However, in modern ES modules, exports are **live bindings**, and a router is an **object passed by reference**. So even `export const welcomeRouter = Router();` followed by routes added later would usually still work, because the importing file receives the same object that later gets its routes. Follow the instructor's convention in your code and exams; just know that the underlying mechanism is a bit more forgiving than 'exported blank'.

7. Registering Routers in index.js

Index.js must know where to look for routing. We register each router as a **middleware** using **app.use()**. If you had fifty routers, you would register all fifty in index.js. That is the only routing-related code left there.

- Import each router from its file (default import, no curly braces).
- **Order matters:** register **express.json()** first so request bodies are parsed; **after it** register the routers.
- **app.use("/", welcomeRouter)** - the path is the prefix. Here it is "/", and the full paths ("/product", "/admin") are inside the router files.

```
// index.js  (FINAL after this lecture - reconstructed from the lecture description)
import express from "express";
import welcomeRouter from "./src/routers/WelcomeRouter.js";
import productRouter from "./src/routers/ProductRouter.js";
import adminRouter from "./src/routers/AdminRouter.js";

const app = express();

app.use(express.json());          // 1) body parser FIRST

app.use("/", welcomeRouter);      // 2) then register routers
app.use("/", productRouter);
app.use("/", adminRouter);

app.listen(7500, () => console.log("Server started on port 7500"));
```

Now your index file is much neater and cleaner. Its job is just to create the Express application and start it. Everything else has moved to different files and folders.

Common mistake: Registering routers **before app.use(express.json())** means request.body will be undefined inside controllers, so destructuring name/price/etc. fails.

Common mistake: Forgetting the import of **sqlConnectionPool** inside a controller does not crash the server; the try/catch returns **500 'Something went wrong'** and the real cause is only in the console. Always read the terminal log.

8. Request Flow and Further Layers

8.1 Current architecture (what we built)

```
Client (browser / app)
      |  HTTP request to a URL
      v
index.js  ----------->  Router  ----------->  Controller  ----------->  Database (MySQL)
 (accepts the request,   (decides which        (takes request,           (runs the query,
  delegates)              controller to call)    generates response)       returns result)

Response goes back:  Database -> Controller -> Client
```

Step by step: the client sends a request to a URL on the server. **index.js** accepts it and delegates to the **router**. The router looks at URL + method and calls the right **controller**. If the controller needs data, it sends a query to the database; the result arrives back at the controller; **the controller sends the response** to the client.

8.2 Real-time (enterprise) architecture

In our code the controller still contains business logic and database code, so it mixes concerns. The instructor explained that in real-time applications the controller should **only** accept the request and generate the response. Extra jobs go into further layers:

```
Client -> index.js -> Router -> Controller -> Service -> DAO -> Database
                                                              |
Client <- Controller <- Service <- DAO  <---------- result -------+
```

Layer	Single responsibility	Should NOT know about
Index	Create the Express app, register middleware/routers, start server	Business logic, SQL
Router	Decide which controller handles a URL + HTTP method	Business logic, SQL
Controller	Accept the request and generate the response	Which database is used; how calculations are done
Service	Business logic: take data, do the operation, return the result	HTTP request/response details, where the data is stored
DAO (Data Access Object)	All database operations	Business rules, HTTP

- **Service example from the lecture:** a service that finds a factorial takes a number as argument, calculates, and returns the factorial. If the number must be stored in a database, the service calls the DAO layer; the service itself does not care about the database.
- **Benefit:** if the underlying database changes, only the DAO functions change. Index, router, controller and service stay unchanged. This is the modular approach.
- Every layer follows the **single responsibility principle**: one layer, one job.
- In this course only the controller layer is implemented (time limit). NestJS gives this architecture out of the box.

Concept / Question	Definition or explanation	Purpose / key difference
Service layer	Folder/layer holding business logic	Keeps controllers thin
DAO (Data Access Object)	Layer that talks directly to the database	DB changes affect only this layer
Single Responsibility Principle	Every layer deals with a single responsibility only	Easier maintenance and testing

8.3 Where does middleware fit?

A student asked how middleware works in this architecture. The instructor answered: middleware sits **before the controller**. The router first calls the middleware. The middleware decides whether the controller should run:

```
Client -> index.js -> Router -> Middleware -> Controller -> ... -> Response
                                    |
                          (invalid / not allowed)
                                      +--> middleware itself sends the error response
```

- If the check passes, the middleware calls **next** and only then is the controller called.
- If the check fails (invalid scenario), the middleware sends the response itself, and the controller never runs.
- In this lecture no middleware had been added yet apart from express.json(); the authentication middleware comes later.

8.4 Trade-off: more layers = more time?

A student raised that more layers increase time complexity. The instructor agreed that processing time increases slightly with more layers, but it is a trade-off: without such an architecture the application suffers badly in **scalability** and **maintainability** and development time grows. The slight performance cost can be handled with better servers. You should accept this trade-off for enterprise applications.

9. Securing APIs: Authentication

9.1 The problem

Currently **all** our APIs are open. The instructor made the point: if he gives you the URLs, anyone in the world can call the product APIs to create, read, update or delete data. That must not happen in any enterprise application. For example, in an e-commerce application the secured APIs should be callable only by the authenticated users of that application.

> We have to add a flow of security: authentication. Only authenticated people can call our API. Everyone else must be rejected with a response such as 'please authenticate first'.

9.2 The idea: login, then proof of login (token)

1. The user first performs a **login** (the front end shows a login form; the back end has a login API).
2. After a **successful** login, the user can access the secured APIs. Until the user has logged in, a call to those APIs must be rejected.
3. So for every secured API (delete / update / create / fetch) the back end must recognise whether the current user is logged in.

9.3 Self-service restaurant analogy

In a self-service restaurant there is a **payment counter** and a separate **food counter**. The secured resource is the **food**. To get the food you first pay (that is your authentication). As proof of payment you receive a **token / receipt**. At the food counter you show the token. The staff do not check your payment history; they only check that you have the token.

Restaurant	Web application
Payment counter	Login API (checks username and password)
Making the payment	Logging in with correct credentials
Token / payment receipt	Token generated by the back end after successful login
Food (secured resource)	Secured APIs: delete, update, create, fetch product
Food counter staff checking the token	Middleware checking that the request contains a token
Customer (you)	Client: website, Android app, iOS app
Customer keeps the token safely; losing it is the customer's fault	Client stores the token and is responsible for it

9.4 Why not sessions?

If front end and back end run on the **same origin / same server**, you can maintain a **session**: start a session on login, terminate it on logout. That was the older style, when applications mixed front end and back end on one server.

Nowadays the front end and back end are **stateless** and run on totally different servers/origins. They communicate only through APIs. So we **cannot maintain a continuous session on the server side**, and instead we use **token-based authentication**.

Point	Session authentication	Token authentication (this lecture)
Where front/back end run	Same origin / same server	Different servers / origins
State	Server keeps a session	Server keeps no continuous state; client carries a token
Login step	Start a session	Generate a token and return it in the response
Logout	Terminate the session	Client discards the token

9.5 The login API flow

1. Front end shows a **login form** (username/phone and password). The user clicks the login button.
2. A request goes to the back end **login API**. The login API reads the username and password from the **request body**.
3. It **compares** them with the values stored in the database (this is the payment counter of your application).
4. If they **match**: login successful. Create a **token** and send it in the response with a success message such as 'login successful'.
5. If they **do not match**: send an error message such as **'invalid credentials'** (no token).

```
POST /login   body: { "phone": "...", "password": "..." }

Match     ->  { "message": "login successful", "token": "<alphanumeric token>" }
No match  ->  { "message": "invalid credentials" }
```

[Extra - not from the lecture] The lecture said 'consider a token here; we will see it later'. The JSON shape above is an illustration only. The actual login API and token generation were **not** built in this session.

9.6 Calling a secured API

1. On a successful login the **client stores the token** (website, Android app, iOS app each keep it in their own memory/storage).
2. When the admin clicks the **Delete** button, the front end calls the delete API and **adds the token to the request**.

3. On the back end, a **middleware** runs before the controller. It checks whether the incoming request contains a valid token.
4. Token present (and valid): the middleware passes control to the **controller**, which executes the delete.
5. Token missing: the middleware itself replies with **'token is missing, user is not authenticated'**; the controller never runs.

```
Login:    Client --(username, password)--> Login API --> DB compare --> token in response
Secured:  Client --(request + token)--> Router --> Middleware --token ok?--> Controller --> Response
                                                          |
                                                          +-- no token --> "token is missing, not authenticated"
```

9.7 Student questions answered

Question	Instructor's answer
Why create a separate login API? Can't middleware do it?	Middleware is for **validating** authentication (checking whether the user is logged in). It does not actually log the user in. First a login API must verify credentials and give the token; then middleware checks that token on later requests. (Example: apps like Gmail and payment apps call a login API on their back end.)
Where do we check 'is the admin logged in'? Front end or back end?	First of all it must be validated at the **back end**, because the back end holds the secured APIs. The server does not otherwise remember who logged in, so the token in the request is the proof.
How does the back end know the user is logged in?	The client sends the token it received at login with each secured request. The middleware reads the request and checks for it.
Who is responsible for the token?	The **client** (customer in the analogy). A lost token is the client's fault; keep it securely.
How is authentication related to front end screens (for example a dashboard)?	The same token is used on the front end: if the user is not logged in, redirect to the login screen. So authentication is applied on **both** front end and back end.

10. Admin Registration API

Before login can exist, there must be a **registration flow**. The instructor created a new entity, **admin**, to store admin data (name, phone, password). Later the admin will log in with phone and password.

10.1 Database table

Created in MySQL: an **id** primary key with auto increment, a **name** of length 255, a **phone** of length 10 (the number the admin will log in with) and a **password** of length 500 (a large maximum on purpose).

> **[Extra - not from the lecture]** SQL written from the instructor's spoken description (exact column types were not shown):

```
CREATE TABLE admin (
    id       INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(255),
    phone    VARCHAR(10),
    password VARCHAR(500)
);
```

10.2 AdminController.js

Purpose: register (save) a new admin. It is the same job as saving a product, so the product save code was copied and adapted.

```
// src/controllers/AdminController.js
import { sqlConnectionPool } from "../configs/DbConfig.js";

export async function save(request, response) {
    try {
        // step 1: get data from client {name:"", price: 6677, description:"", qty:8}
        const { name, phone, password } = request.body;

        // step 2: create a SQL INSERT query
```

```
        // const insertQry = `INSERT into product(name, price, description, qty) VALUES('${name}', ${price}, '$
        // this above approach (concating data directly in the SQL query) is prone to sql injection attack

        const insertQry = `INSERT into admin(name, phone, password) VALUES(?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(insertQry, [name, phone, password]);
        console.log(result.insertId);

        response.send({ message: "admin registered", insertedId: result.insertId });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

- Takes **three** values from the body: name, phone, password. The **id** is auto-incremented, so it is not in the INSERT.
- Three placeholders (?) match the three columns and the three values in the array.
- **Execution flow:** read body, build query, run on the pool, send 'admin registered' with the new id; on failure log and send 500.
- The function is **async** because of **await**. The comments are copied from the product code and still mention product; they are leftovers, not meaningful for admin.

10.3 AdminRouter.js and registration

```
import { Router } from "express";
import { save } from "../controllers/AdminController.js";

const adminRouter = Router();

adminRouter.post("/admin", save);

export default adminRouter;
```

Register it in index.js with **app.use("/", adminRouter)** (shown in section 7), then restart the server.

10.4 Testing in Postman

```
POST  http://localhost:7500/admin
Body -> raw -> JSON:
{
  "name": "Ramesh",
  "phone": "707070",
  "password": "Ramesh123"
}

{ "message": "admin registered", "insertedId": 1 }
```

Then in MySQL: `SELECT * FROM admin;` shows the new row. The instructor later removed the test data with `DELETE FROM admin WHERE id = 1;`. (The phone number and password used were random test values; the spoken password was 'Ramesh 123'.)

What went wrong during the demo, and the lesson: the first request returned **'Something went wrong'** because the SQL connection pool had not been imported in AdminController.js. Fix: add the import and restart. A restart also fixed a later problem where a request seemed to fail (caching/stale server). The instructor pointed out that this is the value of a proper failure response: the server did not simply crash, and the client got a response instead of waiting forever.

> **Note:** Input validation (for example requiring exactly 10 digits for the phone) was **skipped** in this lecture; the instructor said it would be done later.

10.5 The loophole: plain-text passwords

> Never store passwords as plain text. If a database administrator (or an attacker) can open the table, they can see every user's password and log in as anyone.

The instructor asked the class to spot the loophole in the admin table, and the class answered: password stored as plain text. He used a social media example: if someone with database access at Facebook or Instagram could read plain passwords, they could log in to any account. The fix is to **encrypt the password before inserting it** into the database. How to encrypt/hash passwords, and the complete login/authentication flow, were left for the next session ('in the evening').

[Extra - not from the lecture] In practice passwords are **hashed** (one-way, for example with bcrypt) rather than reversibly encrypted. The lecture only said 'encrypt' and did not name a library, so this is for your awareness.

11. Source Coverage Checklist

Checked against the transcript from beginning to end:

Topic in transcript	Covered in section
House analogy, separation of concerns, behaviour unchanged, Express vs NestJS	1
Why not everything in index.js; need for a design pattern	1.4
MVC: model (entities), view (front end), controller	2
index.js role; src folder; controllers, configs, routers, middlewares, models	3
Controller definition; one per entity; welcome controller	4
Moving welcome / product functions; export/import; passing reference; relative paths	5 (Phase 3)
Naming convention (save, getAll, getById, deleteById)	5 (Step 3.3)
DbConfig and connection pool	5 (Step 3.2)
Postman test on port 7500	5 (Step 3.4)
Router layer, express Router, welcome/product routers	5 (Phase 4)
export default, importing without braces	6
express.json() first, app.use for routers	7
Flow diagram; service layer; DAO; single responsibility; NestJS	8.1 - 8.2
Middleware position; time-complexity trade-off Q&A;	8.3 - 8.4
Open APIs problem; login; token; restaurant analogy; session vs stateless	9.1 - 9.4
Login API flow; separate login API question; where to check; token responsibility; front end vs back end	9.5 - 9.7
Admin table, AdminController, AdminRouter, Postman test, import error, plain-text password issue	10

12. Revision Material

12.1 Summary of major topics

- **Separation of concerns**: keep each kind of job in its own place so code is scalable, maintainable, debuggable. Behaviour stays the same.
- **MVC**: Model (entities/data), View (front end), Controller (accepts request, generates response).
- **index.js** only creates the app, registers middleware/routers and starts the server.
- **Controllers** are per entity; **configs** hold DB (and other) configuration; **routers** map URL + method to controller; all inside **src**.
- **Router** object comes from **Router()** of express; routers are registered with **app.use()** after **express.json()**.
- **export default** exports a single value, imported with no braces and any name; declare and configure first, then export at the bottom.
- Enterprise flow: Controller -> Service -> DAO -> DB; each layer has a single responsibility. NestJS provides this out of the box.
- **Authentication**: login API validates credentials and returns a **token**; client stores it and sends it with secured API calls; middleware checks it before the controller.
- Passwords must never be stored as plain text; encrypt/hash before saving.

12.2 Important definitions and syntax

Concept / Question	Definition or explanation	Purpose / key difference
Concern	One type of job in an application	Separate concerns for clarity
MVC	Model View Controller design pattern	Standard way to split code
Model	Representation of an entity and its properties	Shared understanding of data
View	What the user sees (front end)	Presentation
Controller	Function that accepts a request and generates a response	Request/response handling
Router	Layer (express Router object) that maps URLs to controllers	Keeps routes out of index.js
Middleware	Code that runs before the controller and may stop the request	Checks such as authentication
Token	Alphanumeric proof of successful login given to the client	Lets the client call secured APIs
Stateless	Server keeps no continuous session between requests	Front end and back end on different servers
`export const x`	Named export	Import with { x }
`export default x`	Default export, one per file	Import with any name, no braces
`import { Router } from 'express'`	Imports the Router factory	Create router objects
`app.use("/", router)`	Registers a router/middleware	Tells index.js where routes live
`request.body / request.params`	Body JSON / URL path values	Reading client input
`VALUES(?,?,?)`	Parameterised SQL query	Prevents SQL injection

12.3 Key differences

Compare	Difference
Controller vs Service vs DAO	Controller: request/response. Service: business logic. DAO: database access.
app vs Router	app is the whole application (listen, use); Router groups routes of one entity in a separate file.
Named vs default export	Named: many per file, braces, exact name. Default: one per file, no braces, any name.
Session vs token	Session: server-side state, same origin. Token: client carries proof; stateless; different origins.
Login API vs authentication middleware	Login API verifies credentials and issues a token. Middleware verifies the token on later requests.
Passing reference vs calling	welcome = reference passed to Express; welcome() = executes immediately.

12.4 Common mistakes to avoid

Mistake	Correction
Calling controller as **welcome()** in the route	Pass the reference: **welcome**
Forgetting **.js** in ES module import paths	Write the full file name with extension
Wrong relative path (e.g. ./configs instead of ../configs from controllers)	Use ../ to go out of the current folder first
Using **app** inside a router file	Use the object returned by **Router()**
Curly braces when importing a default export	Remove braces; any name works
Registering routers before express.json()	Register express.json() first
Forgetting to import sqlConnectionPool in a new controller	Import it from ../configs/DbConfig.js
Forgetting **async** when using **await**	Add async to the function
Repeating entity name in every function/column	Use save, getAll, getById; columns id, name, price
Putting all controllers in one file	One controller file per entity
Storing plain-text passwords	Encrypt/hash before inserting
Concatenating user input into SQL	Use ? placeholders

12.5 Interview questions and answers

Questions marked (E) go beyond what the instructor said; the rest follow the lecture.

Question	Answer
What is separation of concerns?	Keeping different kinds of jobs (requests, logic, data access, config, routing) in different files/layers so the application is easy to scale, maintain and debug.
What does MVC stand for?	Model, View, Controller. Model = data/entities, View = UI, Controller = accepts request and generates response.
What should index.js contain?	Only creating the Express app, registering middlewares/routers and starting the server.
Why one controller per entity?	Changes for one entity stay in one file, so the code stays modular and organised.
Where should database connection code go and why?	In a configs file (DbConfig.js); it is configuration, not controller or server-start logic.
Why use Router() instead of app in route files?	A separate file has no access to the app variable; Express gives a dedicated Router object for this.
Why is export default used for routers?	To export the router after it has been declared and all routes defined; it exports one value and is imported without braces under any name.
Order of express.json() and routers?	express.json() first, then routers, so bodies are parsed before controllers run.

Question	Answer
What are the Service and DAO layers?	Service holds business logic; DAO (Data Access Object) performs database operations. Controllers call services; services call DAOs.
Which layers change if the database changes?	Only the DAO functions.
Where does middleware run?	Before the controller; it may call next or send the response itself.
Why token authentication instead of sessions?	Front end and back end run on different servers/origins and are stateless; the server cannot keep a continuous session.
Explain the token flow.	Login API verifies credentials, returns a token; client stores it and sends it with each secured request; middleware verifies it before the controller.
Why not store plain-text passwords?	Anyone with database access could read and misuse them; encrypt/hash before storing.
Why use placeholders in SQL?	Concatenating input into SQL is prone to SQL injection; placeholders pass values safely.
(E) Why hash rather than encrypt passwords?	Hashes are one-way, so even the server cannot recover the password; encryption is reversible.
(E) How is the token usually sent?	Commonly in the Authorization header of the HTTP request.

12.6 Practice exercises (solutions not included)

Easy

1. Write the meaning of MVC and list one real-world entity for each part of an e-commerce application.
2. Draw the folder structure of an Express project that has product and customer entities.
3. What is wrong with **app.get("/", welcome())**?

Medium

1. Create **CustomerController.js** with **save** and **getAll**, a **CustomerRouter.js**, and register it in index.js.
2. Move the connection details for a second database into **configs/AnotherDbConfig.js** and import it into a controller.
3. Write the import and export lines for a router using export default, then change the import name in index.js and confirm it still works.
4. Add **updateById** to ProductController and its route in ProductRouter.

Challenging

1. Create a **service** layer for products: controller calls service, service calls a **DAO**; move all SQL to the DAO.
2. Write a middleware that rejects requests without a token header and apply it to the delete route.
3. Add validation to the admin registration API (phone must be exactly 10 digits) before inserting.
4. Design (on paper) the login API: inputs, DB comparison, success response and failure response.

12.7 Final revision checklist

[ ] I can explain separation of concerns using the house analogy.

[ ] I can explain Model, View and Controller and give examples.

[ ] I know what index.js should and should not do.

[ ] I can create a controller file per entity and export its functions.

[ ] I can move DB code into configs/DbConfig.js and import it correctly.

[ ] I know the naming convention (save, getAll, getById, deleteById).

[ ] I can create a router with Router() and map routes to controllers.

- [ ] I understand export default and how to import it.
- [ ] I register express.json() before routers using app.use().
- [ ] I can draw the flow Client -> index -> Router -> Controller -> DB, and the Service/DAO extension.
- [ ] I know where middleware runs and what it does.
- [ ] I can explain the restaurant analogy and token-based authentication.
- [ ] I can explain why the login API and the middleware are different things.
- [ ] I can build and test the admin registration API in Postman.
- [ ] I know why plain-text passwords are dangerous.